

A PROPOSED DESIGN AND IMPLEMENTATION OF SWIFTSTORE: A MULTI-VENDOR E-COMMERCE AND DELIVERY MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

**MAYANK JOSHI 24BCE10858
DEEPALI KUMARI 24BCE11526
SWAPNIL SONI 24BCE10071
SAKSHAM AGARWAL 24BCE10432
KAYUR SHARMA 24BCE10869**

*in partial fulfillment for the award of the degree
of*

**BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING**



VIT[®]
BHOPAL
www.vitbhopal.ac.in

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

**VIT BHOPAL UNIVERSITY
KOTRIKALAN, SEHORE
MADHYA PRADESH - 466114**

April , 2025

VIT BHOPAL UNIVERSITY, KOTHRIKALAN, SEHORE
MADHYA PRADESH – 466114

BONAFIDE CERTIFICATE

Certified that this project report titled “ **SWIFTSTORE - FULL-STACK MULTI-VENDOR MARKETPLACE & DELIVERY MANAGEMENT PLATFORM** ” is the bonafide work of “ **Mayank Joshi** (24BCE10858) , **Deepali Kumari** (24BCE11526), **Swapnil Soni** (24BCE10071) , **Saksham Agarwal** (24BCE10432) , **Kayur Sharma** (24BCE10869) ” who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported at this time does not form part of any other project/ research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROGRAM CHAIR

Dr.Vikas Panthi,
Program Chair CSE CORE
School of Computer Science and Engineering
VIT BHOPAL UNIVERSITY

PROJECT GUIDE

Vijay Kumar Trivedi,
Assistant Professor Junior Engineering
School of Computer Science and Engineering
VIT BHOPAL UNIVERSITY

The Project Exhibition II Examination is held on _____

ACKNOWLEDGEMENT

First and foremost I would like to thank the Lord Almighty for His presence and immense blessings throughout the project work.

I wish to express my heartfelt gratitude to Dr.Vikas Panthi, Head of the Department, School of Computer Science and Engineering for much of his valuable support encouragement in carrying out this work.

I would like to thank my internal guide Mr. Vijay Kumar Trivedi, for continually guiding and actively participating in my project, giving valuable suggestions to complete the project work.

I would like to thank all the technical and teaching staff of School of Computer Science and Engineering, who extended directly or indirectly all support.

Last, but not least, I am deeply indebted to my parents who have been the greatest support while I worked day and night for the project to make it a success.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	List of Abbreviations	1
	List of Figures and Graphs	2
	List of Tables	3
	Abstract	4
1	CHAPTER-1: PROJECT DESCRIPTION AND OUTLINE 1.1 Introduction 1.2 Motivation for the work 1.3 Introduction to the project (Techniques) 1.4 Problem Statement 1.5 Objective of the work 1.6 Organization of the project 1.7 Summary	5-10
2	CHAPTER-2: RELATED WORK INVESTIGATION 2.1 Introduction 2.2 Core area of the project 2.3 Existing Approaches/Methods 2.3.1 Approaches/Methods -1 2.3.2 Approaches/Methods -2 2.3.3 Approaches/Methods -3 2.4 Pros and cons of the stated Approaches 2.5 Issues/observations from investigation 2.6 Summary	11-12

3	CHAPTER-3: REQUIREMENT ARTIFACTS 3.1 Introduction 3.2 Hardware and Software requirements 3.3 Specific Project requirements 3.3.1 Data requirement 3.3.2 Functional requirements 3.3.3 Performance and security requirement 3.3.4 Look and Feel Requirements 3.4 Summary	13- 15
4	CHAPTER-4: DESIGN METHODOLOGY AND ITS NOVELTY 4.1 Methodology and goal 4.2 Functional module design and analysis 4.3 Software Architectural designs 4.4 Subsystem services 4.5 User Interface design 4.6 Summary	16 -18
5	CHAPTER-5: TECHNICAL IMPLEMENTATION & ANALYSIS 5.1 Repository and files 5.2 System Workflow 5.3 System Architectures & Modules 5.4 Example Outputs & Interfaces 5.5 System Features & Results 5.6 Performance Analysis 5.7 Testing & Validation 5.8 Serving & Dashboard 5.9 Summary	19-28

6	CHAPTER-6: PROJECT OUTCOME AND APPLICABILITY 6.1 Outline 6.2 Key implementations of the System 6.3 Significant project outcomes 6.4 Real – world Applicability 6.5 Inference	29-30
7	CHAPTER-7: CONCLUSIONS AND RECOMMENDATION 7.1 Outline 7.2 Limitation/Constraints 7.3 Future Enhancements 7.4 Inference	31-32
	References Appendices Appendix A Appendix B Appendix C Appendix D	33 34-36

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
COD	Cash on Delivery
DB	Database
ETA	Estimated Time of Arrival
GPS	Global Positioning System
HTTP	HyperText Transfer Protocol
JS	JavaScript
JSON	JavaScript Object Notation
MVC	Model View Controller
ORM	Object-Relational Mapping
OTP	One-Time Password
RBAC	Role-Based Access Control
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SQLite	Structured Query Language Lightweight
UI/ UX	User Interface / User Experience
UPX	Unified Payments Interface

LIST OF FIGURES AND GRAPHS

FIGURE NO.	TITLE	PAGE NO.
1	Fig 1.1 — System Architecture of SwiftStore platform	7
2	Fig 1.2 — Data Flow Diagram	8
3	Fig 4.1 — User Interface Design of Dashboard	17
4	Fig 5.1 — System Workflow of SwiftStore	20
5	Fig 5.2 — System Architecture Diagram	21
6	Fig 5.3 — Customer Product Browse Interface	22
7	Fig 5.4 — Cart and Checkout Page	22
8	Fig 5.5 — Vendor Dashboard(Product & Orders)	23
9	Fig 5.6 — Delivery Agent Interface	24
10	Fig 5.7 — Nearby Vendors Map Interface	24
11	Fig 5.8 — Admin Dashboard and Analytics	25
12	Fig 5.9 — Dashboard Interface	28
13	Fig 6.1 — Use Case Flow of Multi – Vendor Marketplace System	29

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1	Table 1.1 — Problem in existing solutions	9
2	Table 1.2 — Objectives of the Project	10
3	Table 2.1 — Comparison of existing Platforms	12
4	Table 3.1 — Hardware Requirements	13
5	Table 3.2 — Software/Library Requirements	13
6	Table 3.3 — Data Summary	14
7	Table 6.1 — Project Outcomes Summary	30
8	Table 7.1 — Future Enhancements Roadmap	32

ABSTRACT

SwiftStore is a full-stack, multi-vendor marketplace and delivery management platform designed to unify customers, vendors, and delivery agents within a single digital ecosystem. The system addresses fragmentation in local commerce by centralizing product discovery, order processing, payment handling, and last-mile delivery into one seamless application. It aims to simplify the entire transaction lifecycle, ensuring efficiency, transparency, and accessibility for all stakeholders involved.

The platform is built using Python with the Flask web framework for backend development, while SQLAlchemy serves as the ORM layer connected to an SQLite database. The frontend is developed using HTML5, CSS3, and JavaScript, with Leaflet.js integrated to enable interactive, map-based vendor discovery through OpenStreetMap. Authentication is managed through Flask sessions and Bcrypt hashing to ensure secure user access. A role-based access control system separates functionality across Customer, Vendor, Delivery Agent, and Admin portals. Payment processing includes both Cash on Delivery and UPI-based transactions through Razorpay integration. Automated email notifications are implemented using SMTP via Flask-Mail or smtplib to keep users informed at key stages such as order confirmation, vendor approval, OTP verification, and delivery completion. Geolocation features utilize the Haversine formula for distance calculation and the Nominatim API for reverse geocoding.

The system successfully demonstrates an end-to-end marketplace workflow, from order placement and vendor approval to delivery partner assignment and OTP-based confirmation. The admin dashboard offers comprehensive oversight with features such as revenue tracking, vendor and delivery agent management, and complaint resolution. Vendors are provided with performance analytics, including sales insights, earnings reports, and commission tracking. The platform's modular design enhances scalability and maintainability, while integrated security mechanisms like OTP verification, encrypted passwords, and session isolation ensure reliable and secure operations.

CHAPTER 1

PROJECT DESCRIPTION AND OUTLINE

1.1 Introduction

In today's digital era, online shopping has become very common due to the widespread use of smartphones and the internet. E-commerce platforms allow users to easily browse, purchase, and receive products without visiting physical stores. However, most existing platforms either focus on large-scale businesses or specific categories like food delivery, leaving local vendors underserved.

Our project, **SwiftStore**, is designed to solve this issue by creating a multi-vendor marketplace platform that connects customers, vendors, delivery agents, and administrators in one system. It allows users to discover nearby vendors, place orders, and track deliveries efficiently. The platform also ensures smooth communication and management of orders from start to end.

1.2 Motivation for the Work

The main reasons for developing SwiftStore are:

- **Lack of digital presence for local vendors:** Small shop owners often cannot afford or manage their own online platforms.
- **Customer inconvenience:** People prefer buying from nearby trusted shops but lack a proper digital system.
- **Poor delivery coordination:** Many systems do not provide proper delivery tracking or management.
- **High commission charges:** Platforms like Amazon and Flipkart charge high fees, which are not suitable for small vendors.
- **Complex existing solutions:** Tools like WooCommerce require technical setup, which is difficult for beginners.

SwiftStore aims to provide a simple, affordable, and complete solution for all these problems.

1.3 Introduction to the Project (Techniques)

The project uses the following technologies and techniques:

- **Flask-based backend**

The backend is developed using Python and Flask, which handles routing, business logic, and server operations.

- **Database management (SQLAlchemy + SQLite)**

SQLAlchemy ORM is used to interact with an SQLite database for storing users, products, and orders.

- **Frontend development**

HTML5, CSS3, and JavaScript are used to design responsive and user-friendly interfaces.

- **Map integration and geolocation**

Leaflet.js with OpenStreetMap is used to show nearby vendors, and the Haversine formula calculates distances.

- **Authentication and security**

User login is managed through Flask sessions, and passwords are secured using Bcrypt hashing.

- **Payment and communication system**

Supports Cash on Delivery and UPI (Razorpay), along with email notifications using SMTP.

- **Data flow pipeline**

User Registration → Product Browsing → Order Placement → Vendor Approval → Delivery Assignment → OTP Verification → Delivery Completion

System Architecture of SwiftStore

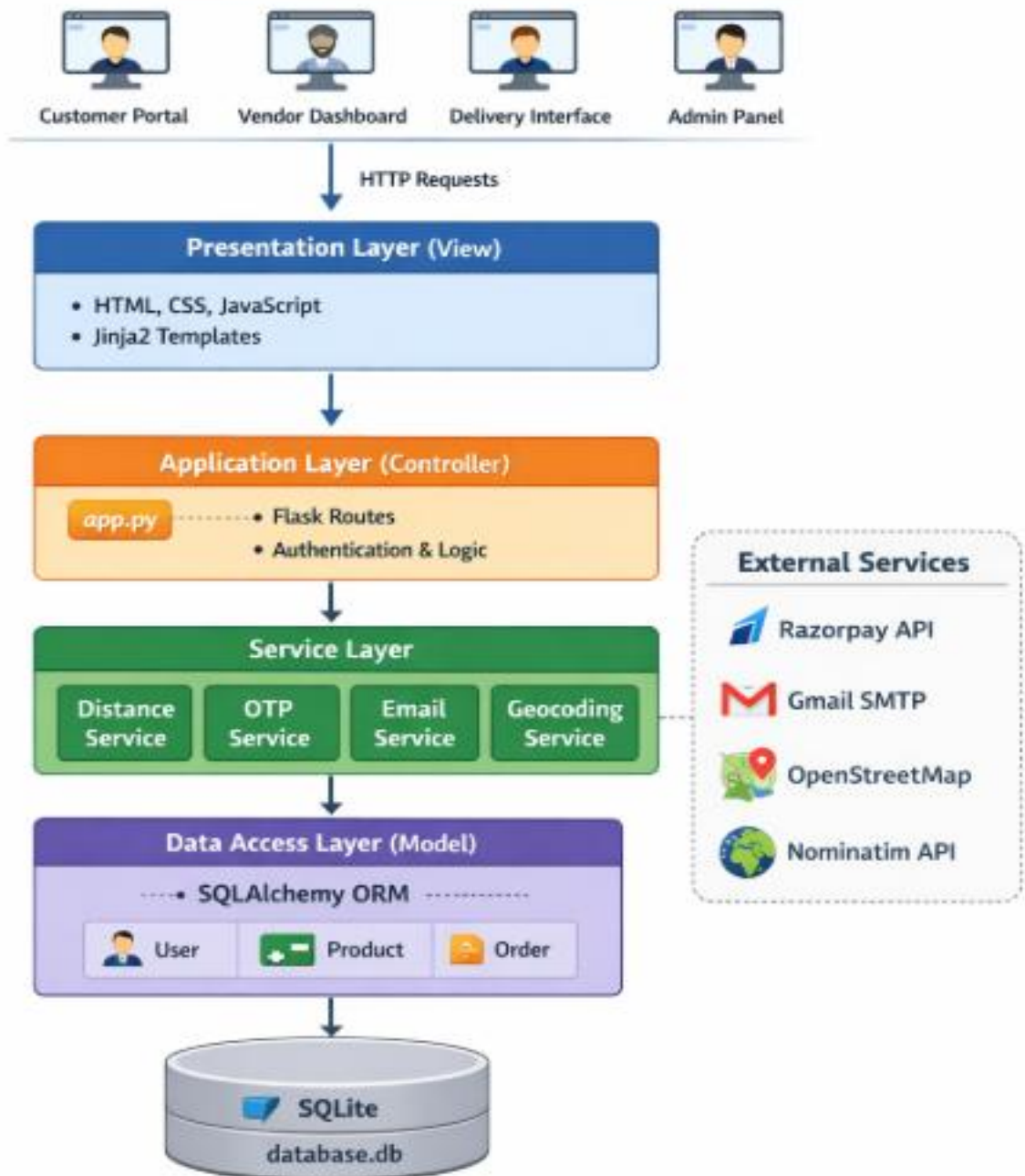


Figure 1.1 — System architecture of SwiftStore platform

Data Flow Diagram



Figure 1.2 — Data Flow Diagram

1.4 Problem Statement

Local vendors and customers face several challenges in adopting digital commerce. Existing platforms are either too expensive, limited to specific categories, or lack proper delivery and communication systems.

- They do not effectively support small local vendors.
- They provide a fragmented shopping experience for customers.
- They lack proper delivery tracking and coordination.
- They rely on manual communication, causing delays and errors.

Problem Statement: To develop a unified multi-vendor platform that connects customers, vendors, and delivery agents, providing efficient product discovery, order management, and reliable delivery with proper tracking and communication.

Table 1.1 — Problems in Existing Systems

Existing Method	Limitations
Amazon/Flipkart	High commission fees
Food Delivery Apps	Limited to specific categories
WhatsApp Business	No delivery tracking or automation
WooCommerce	Complex setup and maintenance

1.5 Objective of the Work

The objectives of this project are:

1. To develop a multi-vendor marketplace platform.
2. To enable efficient order management and delivery tracking.
3. To provide easy product search and nearby vendor discovery.
4. To ensure secure access and smooth system management.

Table 1.2 — Objectives of the Project

Objective	Description
Multi-role System	Separate portals for each user type
Secure Access	Role-based authentication
Order Management	End-to-end order lifecycle handling
Vendor Analytics	Earnings and performance tracking
Delivery System	OTP-based delivery verification
Admin Dashboard	Platform monitoring and control

1.6 Organization of the Project

- **Chapter 2:** Review of existing systems and related work
- **Chapter 3:** Requirements (hardware, software, system needs)
- **Chapter 4:** Design methodology and system architecture
- **Chapter 5:** Implementation details and testing
- **Chapter 6:** Results, outcomes, and real-world applications
- **Chapter 7:** Conclusion, limitations, and future improvements

1.7 Summary

This chapter introduced SwiftStore, a multi-vendor marketplace platform designed to support local vendors and improve the customer shopping experience. The motivation, problem statement, objectives, and system structure were discussed. The project focuses on creating a simple, scalable, and efficient solution using modern web technologies

CHAPTER 2

RELATED WORK INVESTIGATION

2.1 Introduction

E-commerce and delivery platforms have developed rapidly over the years. Early systems mainly focused on simple online shopping without delivery integration. Later, large platforms and specialized delivery apps introduced advanced features like real-time tracking and vendor management. However, most systems are either complex, expensive, or limited to specific categories.

2.2 Core Area of the Project

The main focus of SwiftStore is multi-vendor marketplace management combined with delivery tracking. It integrates vendor product listing, customer ordering, and delivery management in one system. Features like map-based vendor discovery and automated communication improve overall efficiency and user experience.

2.3 Existing Approaches/Methods

2.3.1 Approach 1: Large E-commerce Platforms (Amazon, Flipkart)

- Provide large-scale multi-vendor systems with advanced features.
- Pros: Reliable, feature-rich, strong delivery network.
- Cons: High commission fees and complex onboarding for small vendors.

2.3.2 Approach 2: Food Delivery Platforms (Zomato, Swiggy)

- Focus on real-time delivery and tracking systems.
- Pros: Fast delivery, GPS tracking, OTP verification.
- Cons: Limited to food category, not suitable for general products.

2.3.3 Approach 3: Deep Learning & Hybrid Approaches

- Provide customizable e-commerce solutions using plugins.

- Pros: Flexible and widely used.
- Cons: Complex setup, high maintenance, and dependency issues.

Table 2.1 — Comparison of Existing Platforms

Platform Type	Advantages	Limitations
Amazon/Flipkart	Scalable, feature-rich	High commission, complex onboarding
Zomato/Swiggy	Real-time delivery tracking	Limited to food category
WooCommerce/Magento	Flexible, open-source	Complex setup and maintenance
SwiftStore	Simple, integrated, low cost	Limited scalability (basic version)

2.4 Pros and Cons of Approaches

- Pros: Modern platforms provide advanced features like tracking, automation, and large-scale operations.
- Cons: Most systems are expensive, complex, or not suitable for local vendors and general use.

2.5 Issues/Observations

- Local vendors are not well supported by existing platforms.
- High costs and complexity limit adoption.
- Delivery and marketplace systems are often not integrated.
- Lack of simple and affordable solutions for small businesses.

2.6 Summary

This chapter reviewed existing e-commerce and delivery platforms along with their advantages and limitations. It highlights the need for a simple, integrated, and cost-effective solution like SwiftStore, which combines marketplace and delivery features in one platform.

CHAPTER 3

REQUIREMENT ARTIFACTS

3.1 Introduction

This chapter lists the hardware, software, and system requirements needed to develop the SwiftStore platform.

3.2 Hardware and Software Requirements

Table 3.1 — Hardware Requirements

Component	Specification
Processor	Intel i3 or higher
RAM	4 GB minimum
Storage	10 GB free space
Network	Stable internet connection

Table 3.2 — Software & Library Requirements

Software / Library	Version / Details
Python	3.8+
Flask	2.x
SQLAlchemy	Latest
SQLite	3.x
HTML/CSS/JS	Standard
Leaflet.js	Map integration
Razorpay	Payment integration
smtplib	Email notifications
Git/GitHub	Version control

3.3 Specific Project Requirements

3.3.1 Data Requirements

- User data (Customer, Vendor, Delivery Agent, Admin)
- Product data (name, price, category, stock)
- Order and delivery data (status, payment, OTP)
- Complaint and transaction records

Table 3.3 — Data Summary

Data Type	Description
Users	Stores all user roles and details
Social Sentiment	Vendor product listings
Orders	Order and delivery information
Complaints	Customer issue records

3.3.2 Functional Requirements

- User registration and login with role-based access
- Product browsing and search functionality
- Order placement with payment options (COD/UPI)
- Delivery tracking and OTP-based verification
- Vendor product and order management
- Admin control for users, orders, and complaints

3.3.3 Performance & Security Requirements

- Fast page loading and response time
- Secure password storage using hashing
- OTP verification for delivery security
- Proper session management and access control

3.3.4 Look and Feel Requirements

- Simple and user-friendly interface
- Responsive design for all devices
- Dashboard for each user role
- Clear order status and notifications

3.4 Summary

This chapter presented the complete requirements needed for the development of the SwiftStore platform. It included the hardware and software specifications required to build and run the system efficiently. The chapter also described the important data entities such as users, products, orders, and complaints that are essential for the platform's functioning.

In addition, the functional requirements were defined, covering key features like user registration, product browsing, order placement, delivery tracking, and admin management. The non-functional requirements focused on performance, security, and usability, ensuring that the system is reliable, secure, and easy to use. Overall, this chapter provides a clear foundation for the design and implementation of the SwiftStore system.

CHAPTER 4

DESIGN METHODOLOGY AND ITS NOVELTY

4.1 Methodology and Goal

The design of SwiftStore follows a structured methodology:

1. System design using MVC architecture (Model-View-Controller).
2. Development of user authentication and database models.
3. Implementation of customer, vendor, delivery, and admin modules.
4. Integration of payment, maps, and notification services.
5. Testing and UI improvement.

Goal:

To develop a lightweight and efficient multi-vendor platform that integrates product management, order processing, and delivery tracking in a single system.

4.2 Functional Module Design and Analysis

The system is divided into modules for better organization:

- **Authentication Module:** Handles user registration, login, and role-based access.
- **Customer Module:** Manages product browsing, cart, order placement, and tracking.
- **Vendor Module:** Handles product management and order approval.
- **Delivery Module:** Manages delivery tasks and OTP verification.
- **Admin Module:** Controls users, orders, analytics, and complaints.
- **Notification & Payment Module:** Sends emails and handles payment processing.

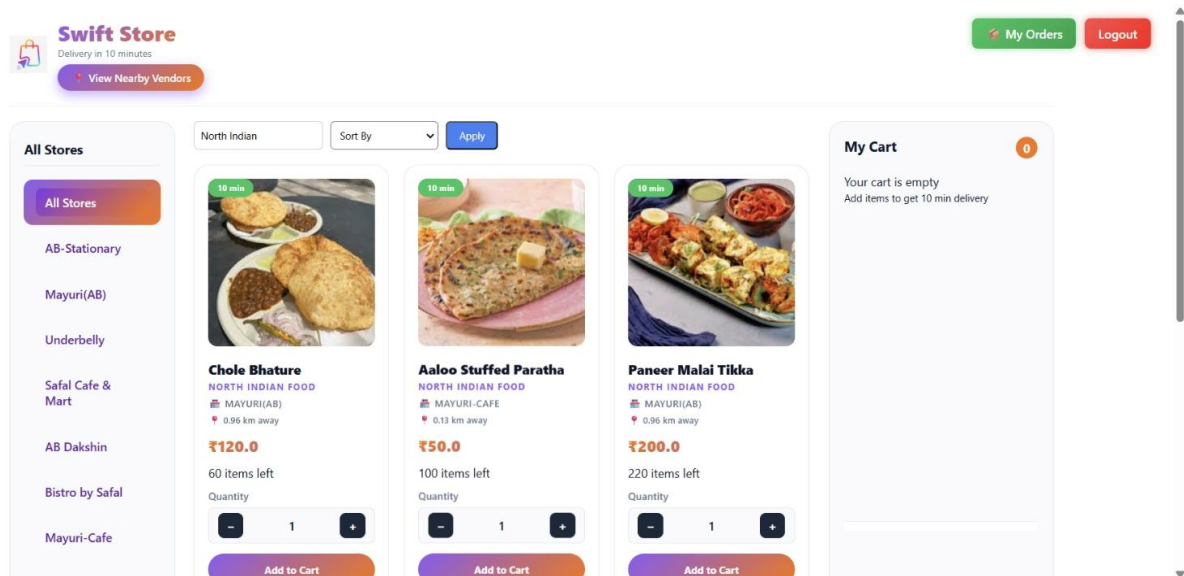
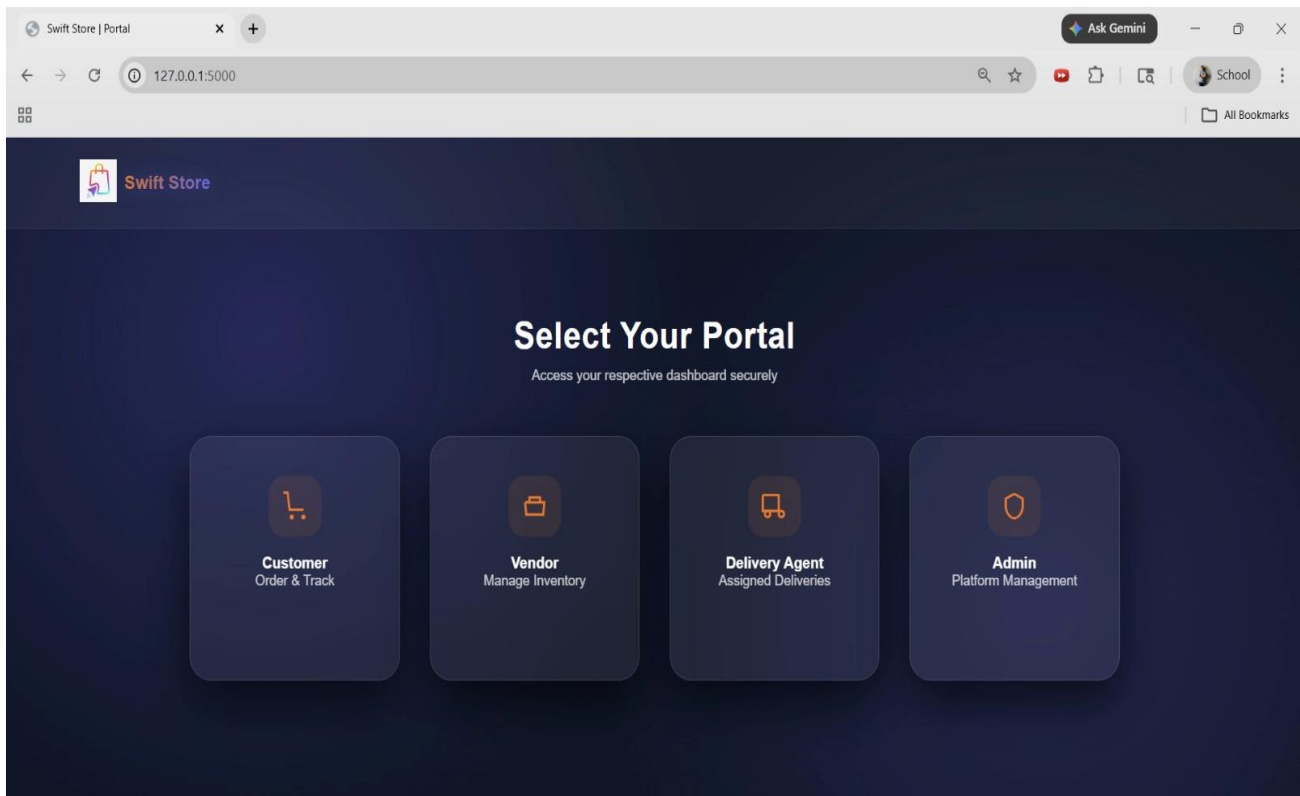


Figure 4.1 — User Interface Design of Dashboard

4.3 Software Architectural Design

The system follows a layered architecture:

- Presentation Layer → User interface (HTML, CSS, JavaScript)
- Application Layer → Flask backend logic and routing
- Data Layer → Database using SQLAlchemy and SQLite

This modular structure makes the system easy to maintain and extend.

4.4 Subsystem Services

- Distance Service: Calculates distance between customer and vendor using formulas.
- OTP Service: Generates and verifies OTP for delivery confirmation.
- Email Service: Sends automated notifications to users.
- Payment Service: Handles online payments and verification.
- Geolocation Service: Provides map-based vendor discovery.

4.5 User Interface Designs

The user interface is designed to be simple and easy to use.

- Separate dashboards for customer, vendor, delivery agent, and admin
- Product browsing with search, filter, and sorting options
- Map view for nearby vendor discovery
- Order tracking with clear status updates
- Responsive design for mobile and desktop devices

4.6 Summary

This chapter explained the design methodology and system architecture of SwiftStore. The project uses a modular and layered approach, making it easy to manage and scalable. The main novelty of the system is the integration of multi-vendor marketplace features with delivery tracking, OTP verification, and map-based discovery in a single platform.

CHAPTER 5

TECHNICAL IMPLEMENTATION & ANALYSIS

5.1 Repository & Files (observed)

The project contains important files such as:

- **app.py** – Main Flask application handling routing and logic
- **database.db** – SQLite database storing all records
- **services/** – Contains helper functions (OTP, email, distance, etc.)
- **templates/** – HTML files for all user portals
- **static/** – CSS, JavaScript, and uploaded images
- **requirements.txt** – List of required libraries

5.2 System Workflow (Detailed)

Steps implemented:

1. User Registration and Login using role-based authentication
2. Vendor adds products with details and images
3. Customer browses products and adds items to cart
4. Order is placed (COD / UPI payment)
5. Vendors approve or reject order items
6. Delivery agent is assigned after approval
7. OTP is generated and sent to customer
8. Delivery completed after OTP verification

SwiftStore System Workflow

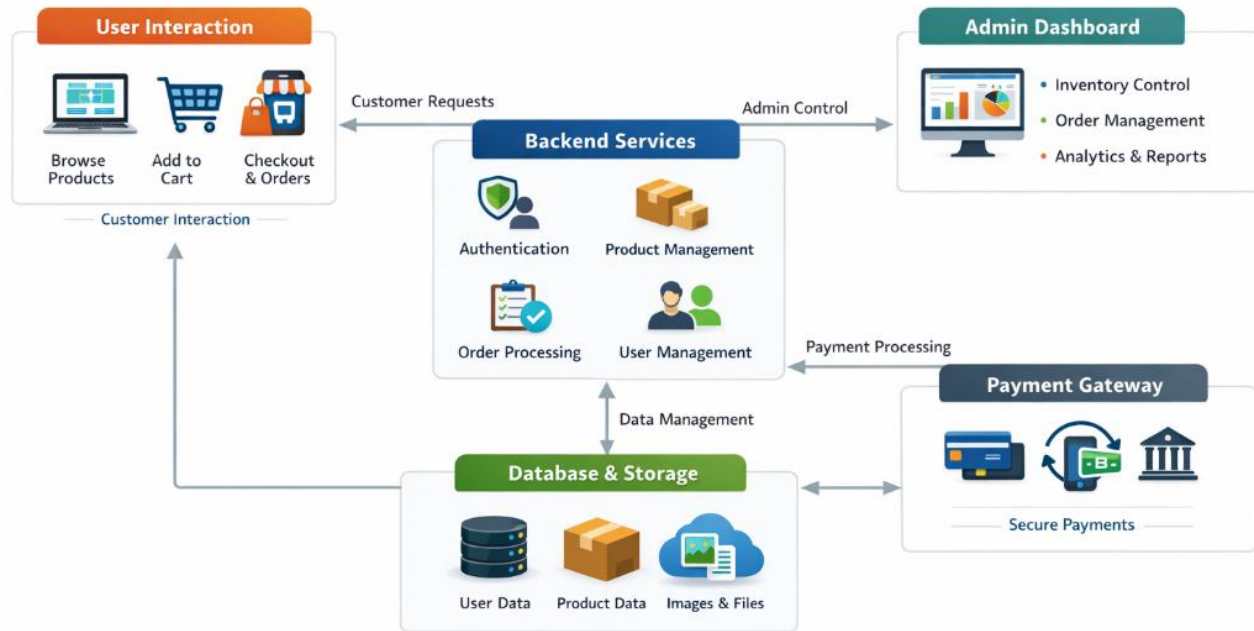


Fig 5.1 – System Workflow of SwiftStore

5.3 System Architecture & Modules

The system is divided into modules:

- Authentication Module – Login, registration, session handling
- Customer Module – Product browsing, cart, orders
- Vendor Module – Product management, order approval
- Delivery Module – Delivery tasks and OTP verification
- Admin Module – User management, analytics, complaints

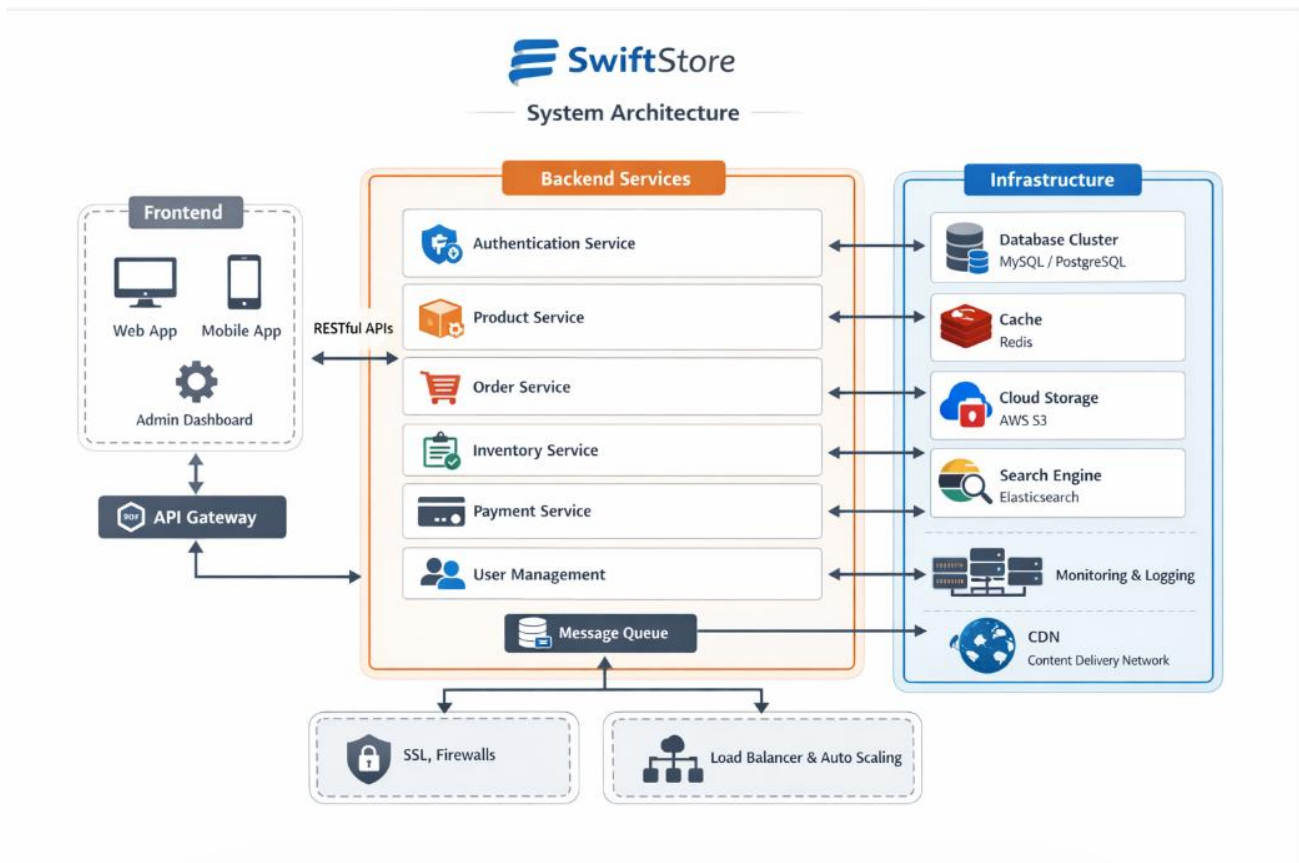


Fig 5.2 – System Architecture Diagram

5.4 Example Outputs & Interfaces

The following outputs were observed during execution:

- Successful login and dashboard redirection
- Product browsing and filtering working correctly
- Orders placed and processed successfully
- OTP verification for delivery working
- Admin able to manage users and complaints

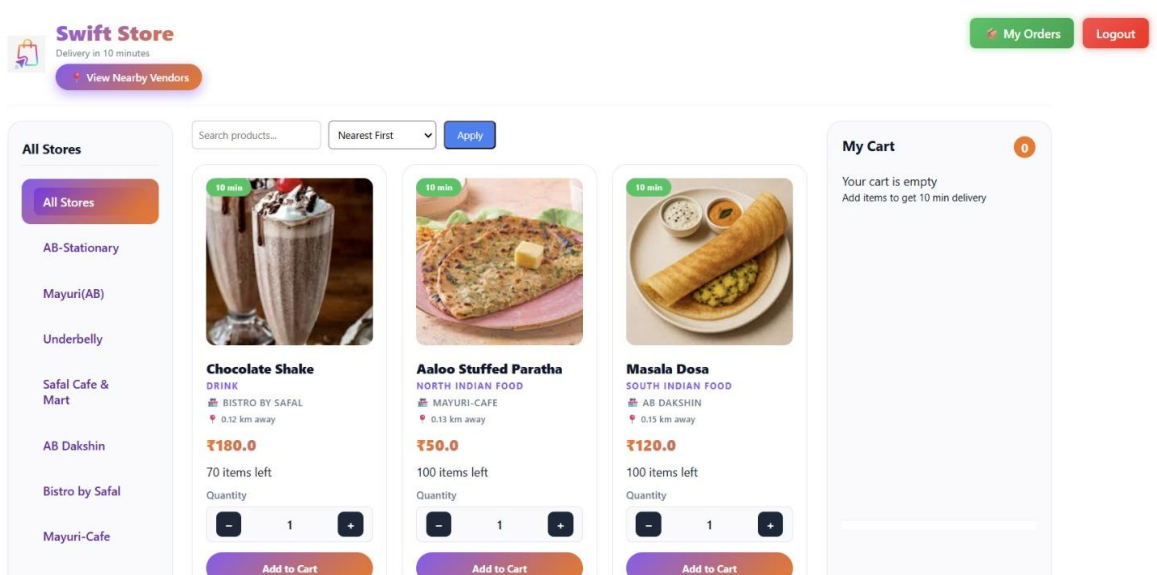


Fig 5.3 — Customer Product Browse Interface

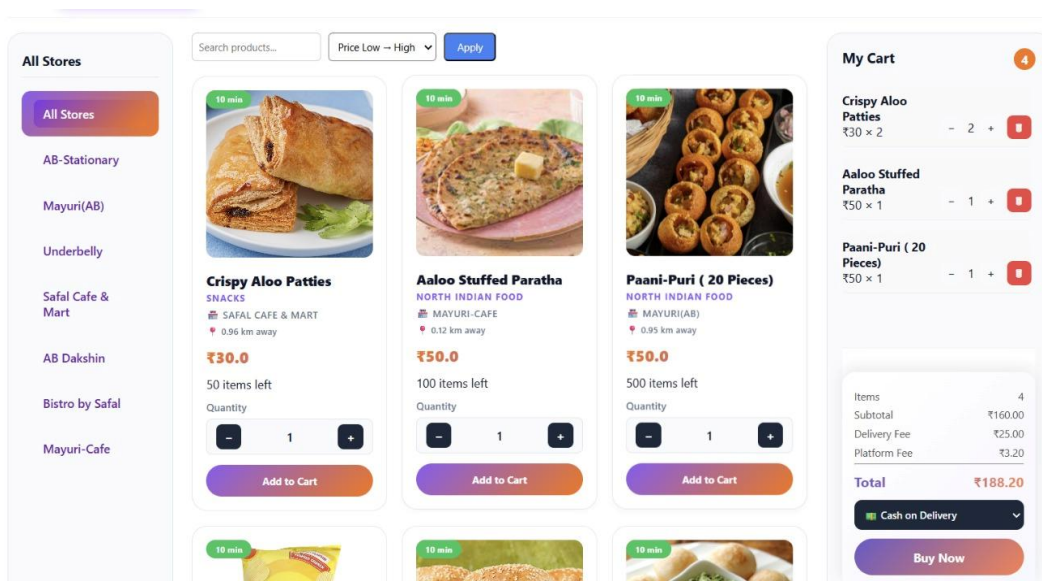


Fig 5.4 — Cart and Checkout Page

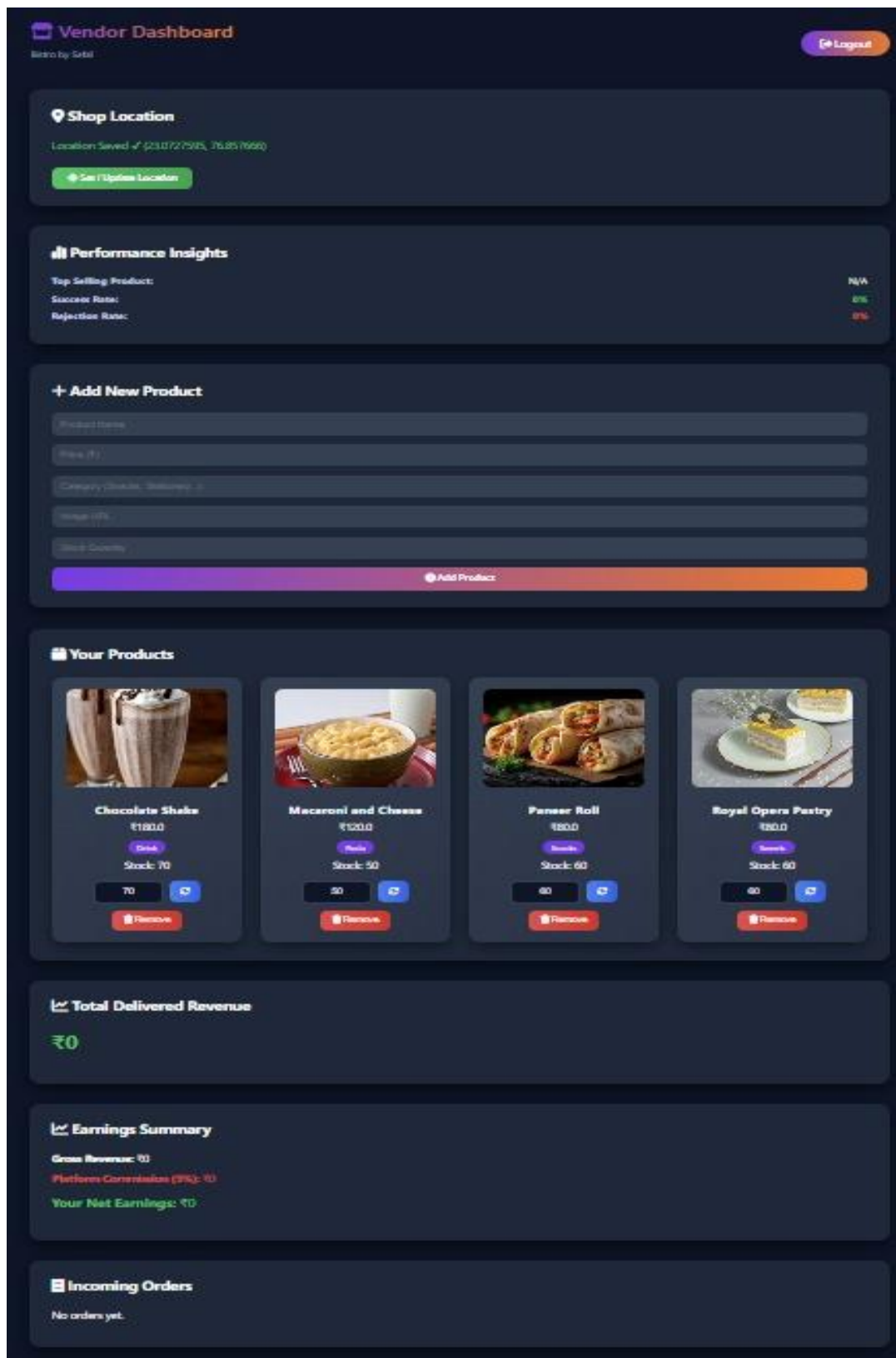


Fig 5.5 — Vendor Dashboard (Product & Orders)

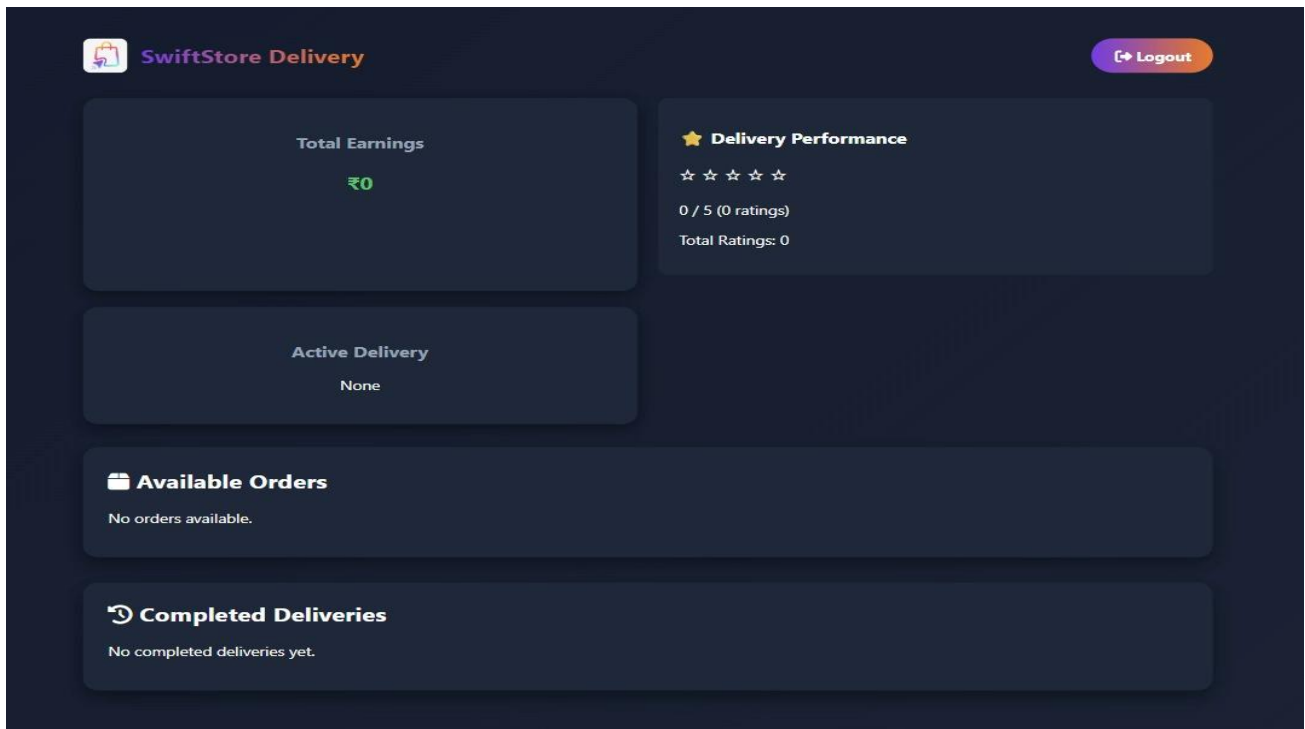


Fig 5.6 — Delivery Agent Interface

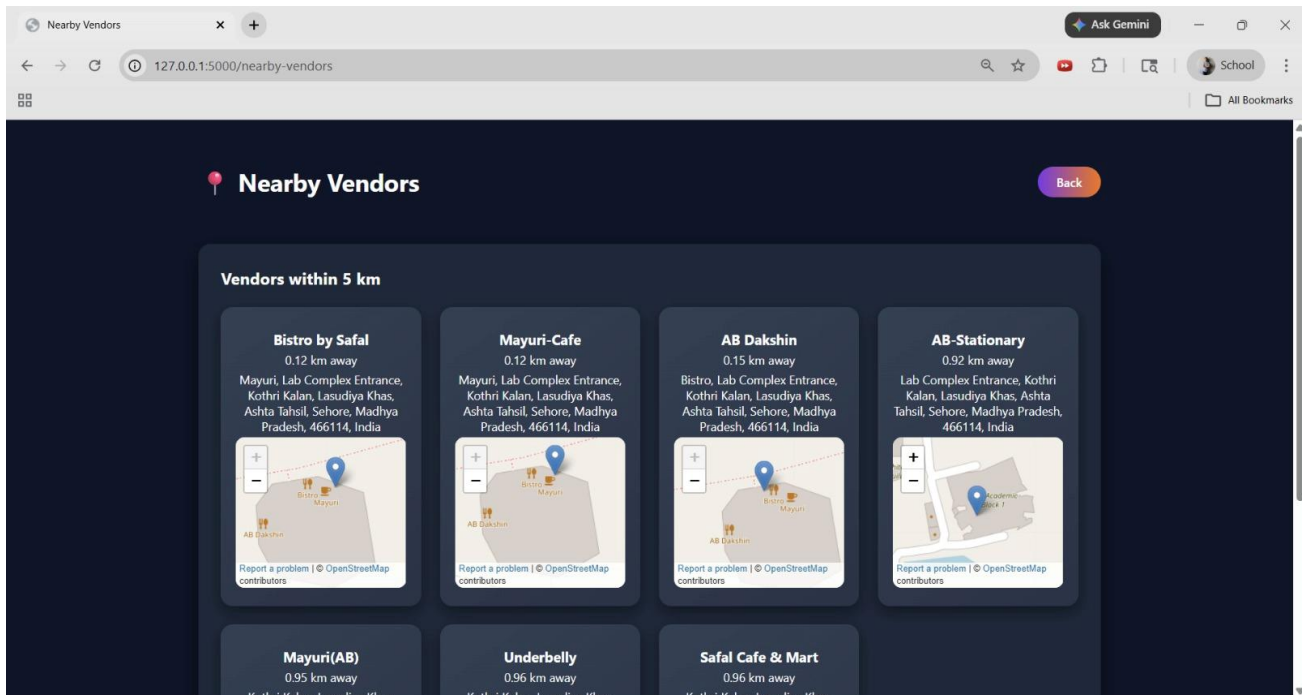


Fig 5.7 — Nearby Vendors Map Interface

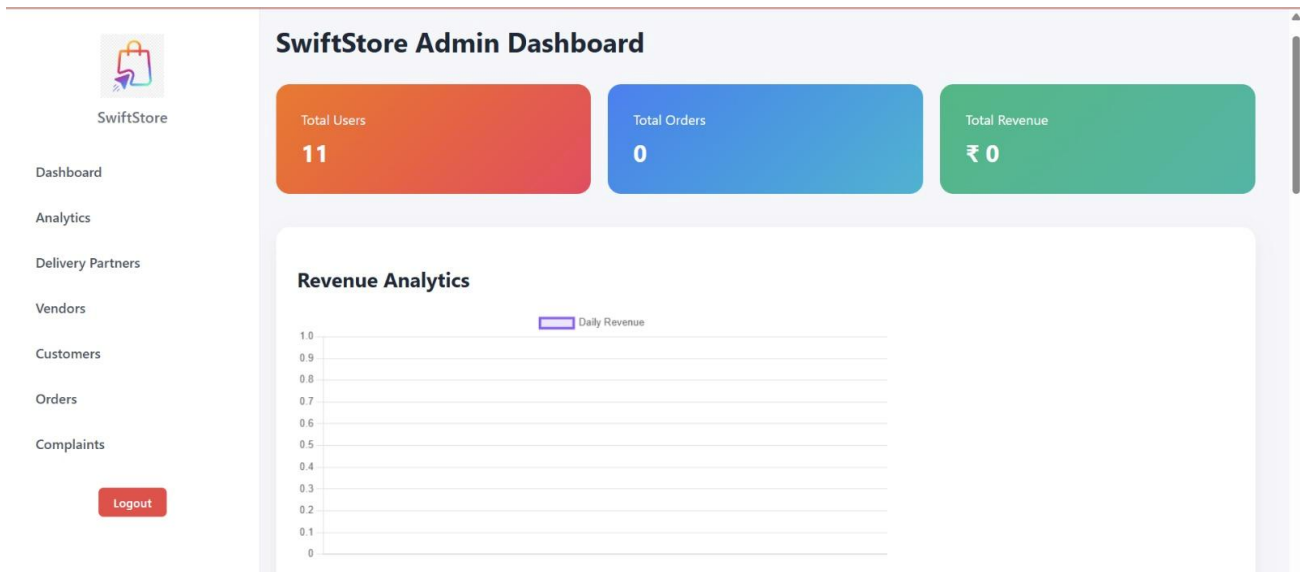


Fig 5.8 — Admin Dashboard and Analytics

5.5 System Features & Results

- Multi-vendor cart system working correctly
- Order approval workflow functioning properly
- OTP-based delivery verification implemented
- Payment system integrated successfully
- Email notifications sent at each stage

5.6 Performance Analysis

Metric	Observation
Page Load Time	< 2 seconds
Database Query Time	Few milliseconds
OTP Generation	Instant
Email Notification	1–3 seconds
System Response	Smooth and reliable

5.7 Testing & Validation

A comprehensive testing process was carried out to ensure that all features of the SwiftStore platform work correctly and efficiently. Different types of testing methods were applied to validate the system functionality.

- **Unit Testing:**

Individual components such as user authentication, OTP generation, and database operations were tested separately to ensure they function correctly.

- **Integration Testing:**

Different modules like customer ordering, vendor approval, and delivery assignment were tested together to verify smooth interaction between them.

- **Functional Testing:**

All major features such as registration, login, product browsing, cart management, order placement, payment processing, and delivery tracking were tested with valid and invalid inputs.

- **OTP Verification Testing:**

Both correct and incorrect OTP cases were tested. The system successfully allowed delivery completion for correct OTP and rejected incorrect entries.

- **User Acceptance Testing (UAT):**

The system interface and usability were tested to ensure that it is user-friendly and easy to understand for all types of users.

Result:

All test cases were successfully executed, and the system performed as expected without major errors.

5.8 Serving & Dashboard

The SwiftStore system is deployed using the Flask web framework, which acts as the backend server handling all client requests and responses. The server processes user inputs, interacts with the database, and renders appropriate web pages dynamically.

The application uses route-based handling where each user role (customer, vendor, delivery agent, admin) is directed to their respective dashboards after login. The server ensures secure session handling and role-based access control.

The dashboard plays an important role in the system by providing a centralized interface for users:

- **Customer Dashboard:**

Allows users to browse products, search items, view nearby vendors on a map, manage cart, place orders, and track order status.

- **Vendor Dashboard:**

Enables vendors to add, update, and delete products, manage inventory, approve or reject orders, and view earnings and analytics.

- **Delivery Agent Dashboard:**

Provides delivery task details, allows status updates, and includes OTP verification for confirming delivery.

- **Admin Dashboard:**

Offers complete control over the system including user management, vendor approval, complaint handling, and platform analytics.

The dashboards are designed using responsive web technologies, ensuring compatibility across desktop and mobile devices.

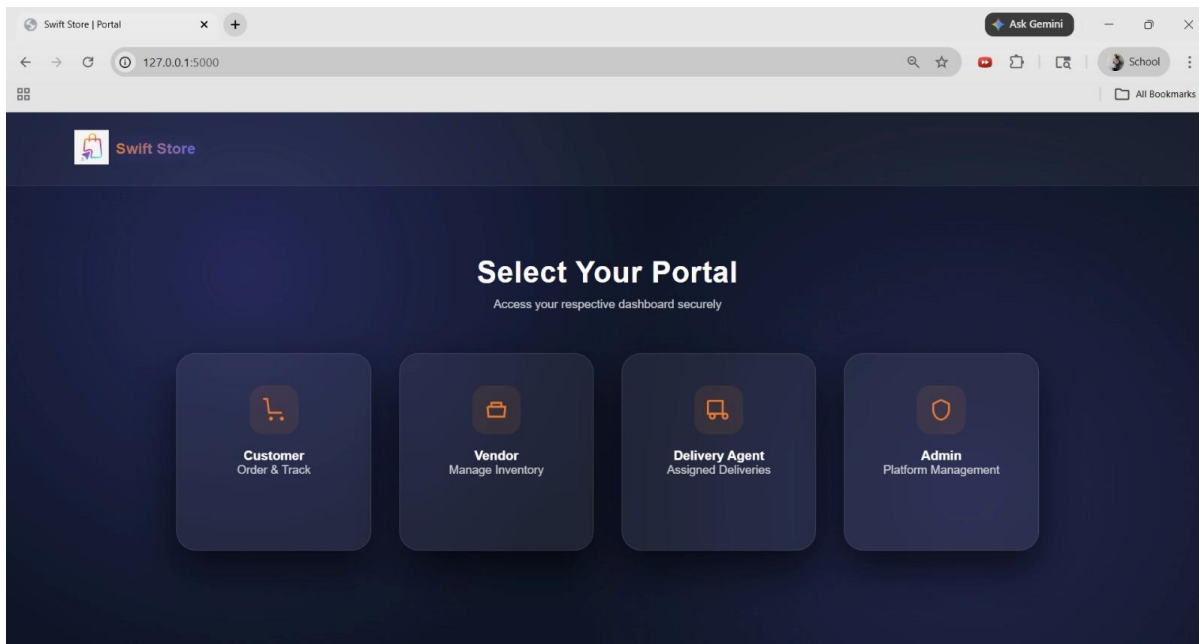


Fig 5.9 — Dashboard Interface

5.9 Summary

This chapter provided a comprehensive overview of the technical implementation and analysis of the SwiftStore platform. It covered the structure of the project repository, system workflow, and the interaction between different modules. The implementation details demonstrated how the system integrates multiple functionalities such as user authentication, product management, order processing, delivery tracking, and payment handling into a single platform.

The chapter also highlighted the different user interfaces designed for customers, vendors, delivery agents, and administrators, ensuring that each role can perform its tasks efficiently. The inclusion of features like OTP-based delivery verification, map-based vendor discovery, and automated email notifications enhances the usability and reliability of the system.

Furthermore, performance analysis confirmed that the system operates efficiently with quick response times and minimal delays. The testing and validation process ensured that all functionalities work correctly under different scenarios, making the system robust and dependable.

Overall, SwiftStore proves to be a well-implemented and effective solution for multi-vendor e-commerce and delivery management, successfully meeting all the objectives defined earlier in the project.

Chapter 6

Project Outcome and Applicability

6.1 Outline

This chapter highlights the main outcomes of the SwiftStore project, its effectiveness, and its applicability in real-world scenarios. It demonstrates how the system fulfills its objectives and provides a practical solution for local multi-vendor commerce.

6.2 Key Implementations of the System

- Multi-role system with Customer, Vendor, Delivery Agent, and Admin portals
- Multi-vendor cart and unified checkout system
- OTP-based secure delivery confirmation
- Automated order lifecycle with email notifications
- Vendor analytics dashboard with earnings and commission tracking
- GPS-based nearby vendor discovery using map integration

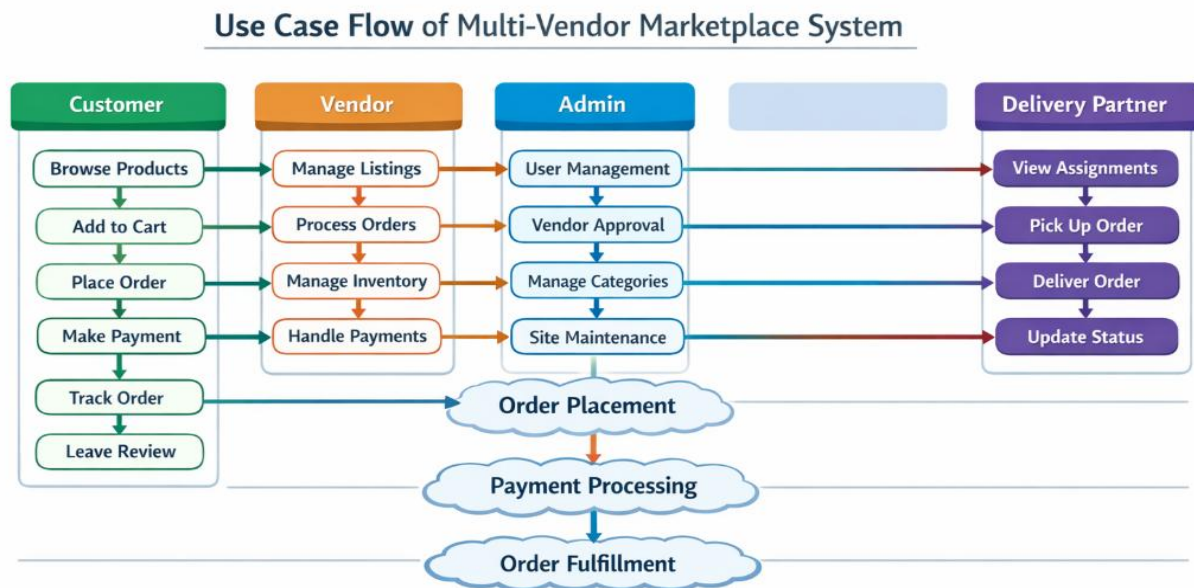


Figure 6.1 — Use Case Flow of Multi-Vendor Marketplace System

6.3 Significant Project Outcomes

- Complete automation of order lifecycle from placement to delivery
- Improved delivery security using OTP verification
- Enhanced vendor transparency with earnings and analytics dashboard

Table 6.1 — Project Outcomes Summary

Outcome	Description
Unified Platform	Connects customers, vendors, and delivery agents in one system
Automation	Fully automated order processing and communication
Security	OTP-based delivery confirmation ensures trust
Vendor Insights	Provides clear earnings and commission breakdown

6.4 Real-world Applicability

- Can be used in local neighborhood markets for small vendors
- Suitable for college campuses and student marketplaces
- Useful for events, exhibitions, and multi-vendor fairs
- Can be deployed in Tier-2 and Tier-3 cities for digital commerce

6.5 Inference

The results of SwiftStore show that a lightweight multi-vendor marketplace with integrated delivery management can be successfully developed using modern web technologies. The platform improves accessibility for local vendors, enhances customer convenience, and ensures secure and efficient order handling. This project proves that such systems can be practically implemented and scaled for real-world applications.

Chapter 7

Conclusions and Recommendations

7.1 Outline

This chapter summarizes the overall results of the SwiftStore project, highlights its limitations, and suggests possible improvements for future development.

7.2 Limitations/Constraints

- The system uses SQLite, which is not suitable for high-traffic and large-scale deployments.
- The platform is web-based and does not have a native mobile application.
- No real-time GPS tracking of delivery agents is available.
- Limited features such as no recommendation system or vendor rating mechanism.

7.3 Future Enhancements

Some possible improvements for the system are:

- Development of a mobile application for better user experience.
- Implementation of real-time GPS tracking for delivery monitoring.
- Integration of advanced features like product recommendations and vendor ratings.

Table 7.1 — Future Enhancements Roadmap

Enhancement	Expected Benefit
Mobile Application	Better user experience and accessibility
Real-time GPS Tracking	Improved delivery transparency
Recommendation System	Personalized user experience
Multi-Database Support	Better scalability and performance

7.4 Inference

SwiftStore successfully demonstrates how a multi-vendor marketplace with integrated delivery management can be built using modern web technologies. The system provides a practical solution for local vendors and improves customer convenience. It also serves as a strong foundation for future improvements and real-world deployment.

REFERENCES

1. P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” National Institute of Standards and Technology, Special Publication 800-145, Sep. 2011.
2. M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed., O’Reilly Media, 2018.
3. A. Ronacher, “Flask — A Lightweight WSGI Micro-Framework,” Pallets Project, 2010.
4. SQLAlchemy Documentation, “The Python SQL Toolkit and Object Relational Mapper,” 2006.
5. R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
6. W. S. Fulton and M. Spiering, “VADER: Valence Aware Dictionary for sEntiment Reasoning,” AAAI, 2014.
7. OpenStreetMap Contributors, “OpenStreetMap,” 2004.
8. Leaflet.js Documentation, “Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps,” 2010.
9. Nominatim Documentation, “Nominatim: Geocoding with OpenStreetMap Data.”
10. Razorpay Developer Documentation, “Razorpay Payment Gateway API Reference,” 2021.
11. Python Documentation, “secrets — Generate Secure Random Numbers for Managing Secrets,” Python 3.8.
12. Bcrypt Documentation, “Flask-Bcrypt: Bcrypt hashing utilities for Flask applications.”
13. S. Srinath, “SMACNA: A Framework for Understanding Digital Marketplace Evolution,” *Journal of E-Commerce Research*, vol. 14, no. 2, pp. 45–62, 2020.
14. W. Sinnott, “Virtues of the Haversine,” *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.
15. Razorpay Python SDK, “razorpay-python: Razorpay Python Client Library,” GitHub.

APPENDICES

Appendix A — Installation & Setup

Prerequisites:

- Python 3.8 or above installed
- pip package manager
- Git installed
- Internet connection
- Gmail account with App Password enabled

Installation Steps:

1. Clone repository:

```
git clone https://github.com/Swapnil-code-maker/Swiftstore.git
cd Swiftstore
```

2. Create virtual environment:

```
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
```

3. Install dependencies:

```
pip install flask flask-sqlalchemy flask-bcrypt razorpay requests
```

4. Set environment variables:

```
export SWIFTSTORE_EMAIL_USER="your_email"
export SWIFTSTORE_EMAIL_PASSWORD="your_password"
export RAZORPAY_KEY_ID="your_key"
export RAZORPAY_KEY_SECRET="your_secret"
export SECRET_KEY="your_secret_key"
```

5. Initialize database:

```
python -c "from app import db; db.create_all()"
```


6. Run application:

```
python app.py
```

```
# Open http://127.0.0.1:5000
```

Appendix B — Directory Structure

Swiftstore/

```
|— app.py
|— database.db
|— services/
|   |— distance.py
|   |— email_service.py
|   |— otp_service.py
|   |— geocoding.py
|— static/
|   |— css/
|   |— js/
|   |— uploads/
|— templates/
|   |— customer/
|   |— vendor/
|   |— delivery/
|   |— admin/
```

Appendix C — Key Code Modules

- Flask application (routing and logic in app.py)
- OTP generation and verification (otp_service.py)
- Email notification system (email_service.py)
- Distance calculation using Haversine formula (distance.py)
- Geolocation and map services (geocoding.py)

Appendix D — Suggested Figures & Where to Capture

1. **Run the application → Capture System Workflow diagram**
(Fig 5.1 — *System Workflow of SwiftStore*)
2. **Capture overall architecture diagram of the system**
(Fig 5.2 — *System Architecture Diagram*)
3. **Login as Customer → Browse products page**
(Fig 5.3 — *Customer Product Browse Interface*)
4. **Add products to cart → Checkout page**
(Fig 5.4 — *Cart and Checkout Page*)
5. **Login as Vendor → Open dashboard (products & orders section)**
(Fig 5.5 — *Vendor Dashboard (Product & Orders)*)
6. **Login as Delivery Agent → View delivery interface**
(Fig 5.6 — *Delivery Agent Interface*)
7. **Open Nearby Vendors Map page**
(Fig 5.7 — *Nearby Vendors Map Interface*)
8. **Login as Admin → Open admin dashboard with analytics**
(Fig 5.8 — *Admin Dashboard and Analytics*)
9. **Capture dashboard overview interface (main UI view)**
(Fig 5.9 — *Dashboard Interface*)